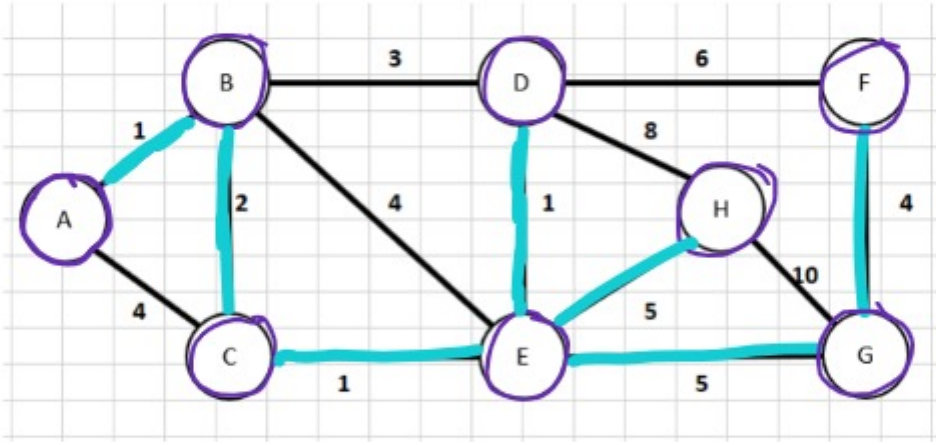
Katie's algo playground

- Lejátszani:
  - Prim,
  - Kruskal,
  - Dijkstra,
  - DAG-os,
  - Bellman-Ford,
  - Huffman,
  - LZW
- Struki:
  - Valamilyen legrövidebb utas/sima gráfos dolog

Prim algoritmus

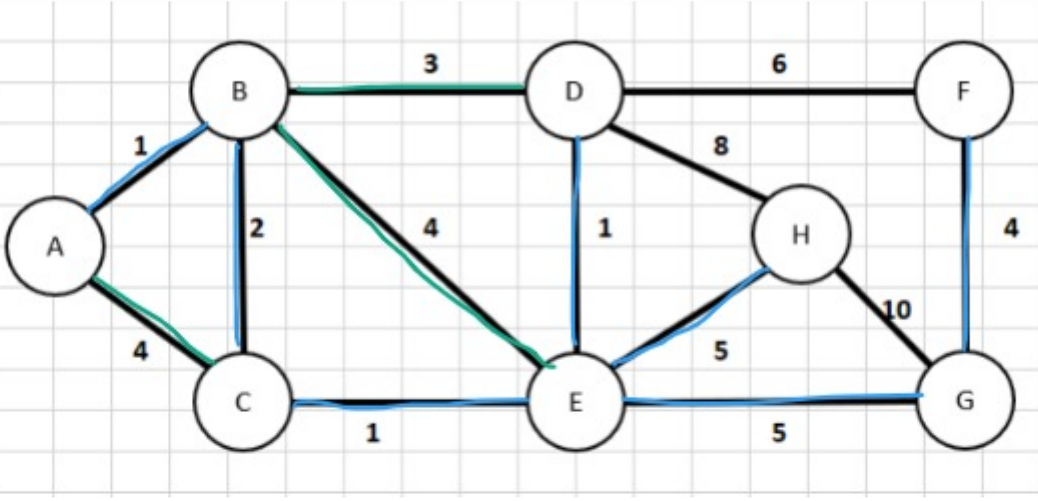
- Gráfból feszítőfát építünk
  - Az alapja a szélességi bejárás
  - c: költség (cost)
  - p: szülő (parent)
- Algoritmus "konyhanyelven"
  - A legkönnyebb csúcstól indulunk
  - Felírjuk a c-ben a szomszédaihoz tartozó élsúlyt és p-hez a vizsgált csúcsot
  - A kisebb élsúly (c) felé haladunk
  - Felírjuk az élsúlyokat az újonnan vizsgált csúcs szomszédaihoz
    - Ha már vizsgálva volt a csúcs, akkor oda nem írunk
    - Ha írunk már korábban c-t a szomszédhoz és a jelenleg vizsgált csúcsból kisebb az élsúly, akkor beírjuk (a súlyt és az új szülőt is). Ha nagyobb lenne az új, akkor nem!
    - Ezt addig folytatjuk, amíg végig nem érünk az összes csúcson.



c értékek a Q-ban								Fába kerülő csúcs	p értékek változásai							
A	B	C	D	E	F	G	H		A	B	C	D	E	F	G	H
0	∞	∞	∞	∞	∞	∞	∞		0	0	0	0	0	0	0	0
	1	4						A		A	A					
		2	3	4				B			B	B	B			
				1				C					C			
			1			5	5	E				E			E	E
					6			D						D		
					4			G						G		
								F								
								H								
0	1	2	1	1	4	5	5		0	A	B	E	C	G	E	E

Kruskal algoritmus

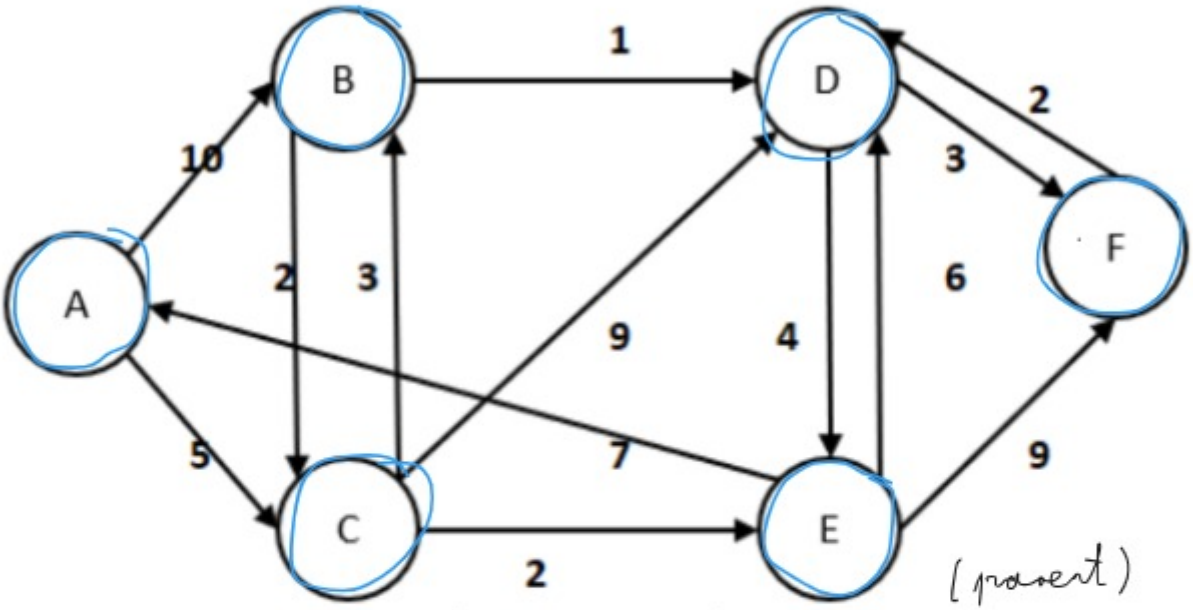
- Halmazokat kreálunk az élekből és a csúcsokból
- Algoritmus "konyhanyelven"
  - Felírjuk az életeket (pl. (A,B)) és az élsúlyokat
  - Sorrendbe tesszük az élsúlyok szerint és ebben a sorrendben haladunk
  - Az élekből halmazokat képzünk
  - Ha olyan él jön, amelynek valamelyik csúcsa már része valamelyik halmaznak, akkor az élhez tartozó másik csúcsot is ahhoz a halmazhoz tesszük
  - Ha olyan él jön, amelynek minkét csúcsa egy-egy külön halmazban van, akkor a halmazt uniózzuk
  - Ha az összes egy halmazban van, akkor terminálunk



			{A}	{B}	{C}	{D}	{E}	{F}	{G}	{H}
(A,B)	1		(A, B)	{C}	{D}	{E}	{F}	{G}	{H}	
(C,E)	1		(A, B)	(C, E)	{D}	{F}	{G}	{H}		
(D,E)	1		(A, B)	(C, D, E)	{F}	{G}	{H}			
(B,C)	2		(A, B, C, D, E)	{F}	{G}	{H}				
(B,D)	3	x								
(A,C)	4	x								
(B,E)	4	x								
(F,G)	4		(A, B, C, D, E)	(F, G)	{H}					
(E,H)	5		(A, B, C, D, E, H)	(F, G)						
(E,G)	5		(A, B, C, D, E, F, G, H)							
(D,F)	6		terminálunk							
(D,H)	8									
(H,G)	10									

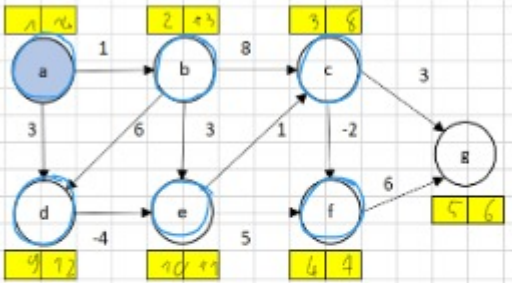
Dijkstra algoritmus

- Gráfból feszítőfát építünk
  - d: távolság
  - p: szülő (parent)
  - Előfeltétele, hogy a gráfban nem lehet negatív súlyú él
- Algoritmus "konyhanyelven"
  - Megnézzük, hogy honnan indulunk a legkevesebbel
  - Onnantól felírjuk, hogy a kezdő csúcsból mennyivel jutunk el az adott csúcsig (számítva az odáig vezető élsúlyokat is)
    - pl. a 8* esetén a 3 + 5 = 8
  - Mindig arra haladunk, amelyik d-je a legkisebb (min prioritásos sor)
  - Ha kisebb lesz az út abból az irányból, akkor update a d-re és a p-re is
  - Addig megyünk, amíg az összes csúcson végig nem érünk



d értékek a Q-ban						Kiterjesztett	p értékek változásai					
A	B	C	D	E	F	csúcs	A	B	C	D	E	F
0	∞	∞	∞	∞	∞	u:d(u)	0	0	0	0	0	0
·	10	5				A		A	A			
·	8	·	14	7		C : 5		C		C	C	
·	·	·	13	·	16	E : 7				E		F
·	·	·	9	·		B : 8				B		
·	·	·	·	·	12	D : 9						D
						F : 12						

DAG (lehet negatív élsúlyunk) (mélységi bejárás az alapja)



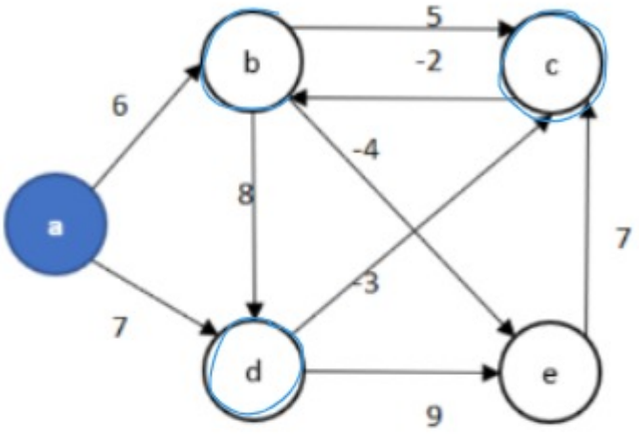
	d értékek változásai							Kiterjesztett	p értékek változásai						
	a	b	c	d	e	f	g	csúcs	a	b	c	d	e	f	g
	0	∞	∞	∞	∞	∞	∞	u:d(u)	0	0	0	0	0	0	0
a	•	1		3				a : 0		a		a			
b	•	•	9	•	4			b : 1			b		b		
d	•	•		•	-1			d : 3					d		
e	•	•	0	•	•	4		e : -1			e			e	
c	•	•	•	•	•	-2	3	c : 0						c	c
f	•	•	•	•	•	•	•	f : -2							
g								g : 3							
	0	1	0	3	-1	-2	3		0	a	e	a	d	c	c

Stack (legutolsó d)

- Gráfból feszítőfát építünk
  - d: távolság
  - p: szülő
  - Megengedi a negatív élsúlyokat!
- Algoritmus "konyhanyelven"
  - Mélységi bejárással kezdjük (ekkor még nem vesszük figyelembe az élsúlyokat)
  - Felírjuk a stack-et és az alapján haladunk (az érték a táblázatbeli)
  - Ezután kb. mintha Dijkstra-t csinálnánk

Bellmann-Ford algoritmus

- Gráfból feszítőfát építünk
  - d: távolság
  - e: lépés szám
  - p: szülő (parent)
  - Negatív kör, ha e > csúcsok száma
  - Soron alapul
- Algoritmus "konyhanyelven"
  - kiterjesztés: kimásolni amit épp nézünk
  - menet: adott e + 1
  - A sor alapján haladunk



távolság · lépések száma · (másol) · parent · adott e + 1

d(); e() változásai						kiterjesztés: d();e()	Q	p() változásai						menet
a	b	c	d	e				a	b	c	d	e		
0;0	∞	∞	∞	∞			<a>	0	0	0	0	0		
	6;1		7;1			a : 0,0	<b,d>		a		a			1
		11;2		2;2		b : 6,1	<d,c,e>			b		b		2
		4;2				d : 7;1	<c,e>			d				2
	2;3					c : 4;2	<e,b>		c					3
						e : 2;2	<b>							3
				-2;4		b : 2;3	<e>					b		4
						e : -2;4	<>							5
0;0	2;3	4;2	7;1	-2;4				0	c	d	a	b		

Megoldás

d(); e() változásai					kiterjesztés: d();e()	Q	p() változásai					menet
a	b	c	d	e			a	b	c	d	e	
0;0	∞	∞	∞	∞		<a>	0	0	0	0	0	
	6;1		7;1		a: 0;0	<b,d>		a		a		1
		11;2		2;2	b: 6;1	<d,c,e>			b		b	2
		4;2			d: 7;1	<c,e>			d			2
	2;3				c: 4;2	<e,b>		c				3
					e: 2;2	<b>						3
				-2;4	b: 2;3	<e>					b	4
					e: -2;4	<>						5

0;0	2;3	4;2	7;1	-2;4			0	c	d	a	b	

1. feladat: (10 pont)
Tömörítse **Huffman algoritmussal** az alábbi szöveget. Építse fel a **kódfát**, határozza meg az **ábécé betűinek kódját**. Adja meg naiv tömörítéssel és a Huffman kódfa alapján kapott tömörített hosszt bitekben. (Naiv tömörítés esetén a jel hosszának azt a legkisebb bitszámot kell tekinteni, amin az ábécé betűi megkülönböztethetők.)

TÖRÖKÖK HÁNY TÖRÖTT TÖRÖK TÖRT TÖRTEK EL

Karaktereket összeszámolni

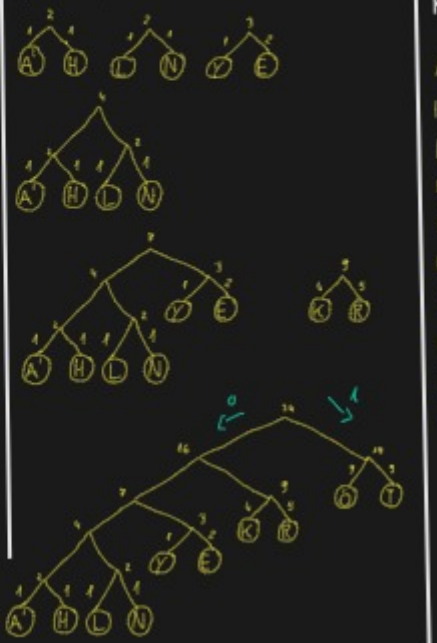
T: 3
Ö: 3
R: 5
K: 4
H: 1
A': 1
N: 1
Y: 1
E: 2
L: 1

Sorrendbe tenni

A': 1
H: 1
L: 1
N: 1
Y: 1
E: 2
K: 4
R: 5
Ö: 3
T: 3

AHL
LN 2
YE 3
KR 9
ÖT 18

Fát készítünk belőle



Kiírjuk a kódot

A': 00000
H: 00001
L: 0010
N: 0011
Y: 010
E: 011
K: 10
R: 11
Ö: 10
T: 11

Huffman alapján a hossz

5·1·4 + 4·1·1 + 4·2 +
+ 3·4 + 3·5 + 2·9·2 =
= 20 + 4 + 8 + 12 + 15 + 36 = 95

Naiv alapján

10 féle dolgot kiírnivan

· 34 3

$\overline{2} \cdot \overline{2} \cdot \overline{2} \cdot \overline{2} = 16$

10 féle dolgot

bitről van